

TimeAudit — 你这台 Windows 电脑的"黑匣子"

一个 7×24 小时在后台默默运行的个人遥测系统：每 3 秒给你的电脑拍一张"全身 X 光"，把"哪个窗口在用、每个进程吃了多少 CPU/显卡/内存/硬盘/网络、整机温度功耗帧率、哪个进程刚出生/刚崩溃"全部记进数据库，然后用网页大盘 (Grafana) 回放出来。

这份 README 写给两类读者：

- **人类开发者**：哪怕你没接触过这个项目，也能在 20 分钟内搞懂它"是什么、怎么转、每个文件干嘛"。
- **AI 维护者**（比如接手改代码的 Claude）：文末有"关键不变量 / 千万别踩的坑 / 测试入口"，先读那部分再动手。

想直接装起来 / 换电脑 / 灾后恢复？看 [快速部署.md](#)。想看懂 6 张仪表盘的**每一个面板**、学会用数据判断电脑性能与问题？看 [使用手册.md](#)（面向完全小白，78 个面板逐个精讲）。

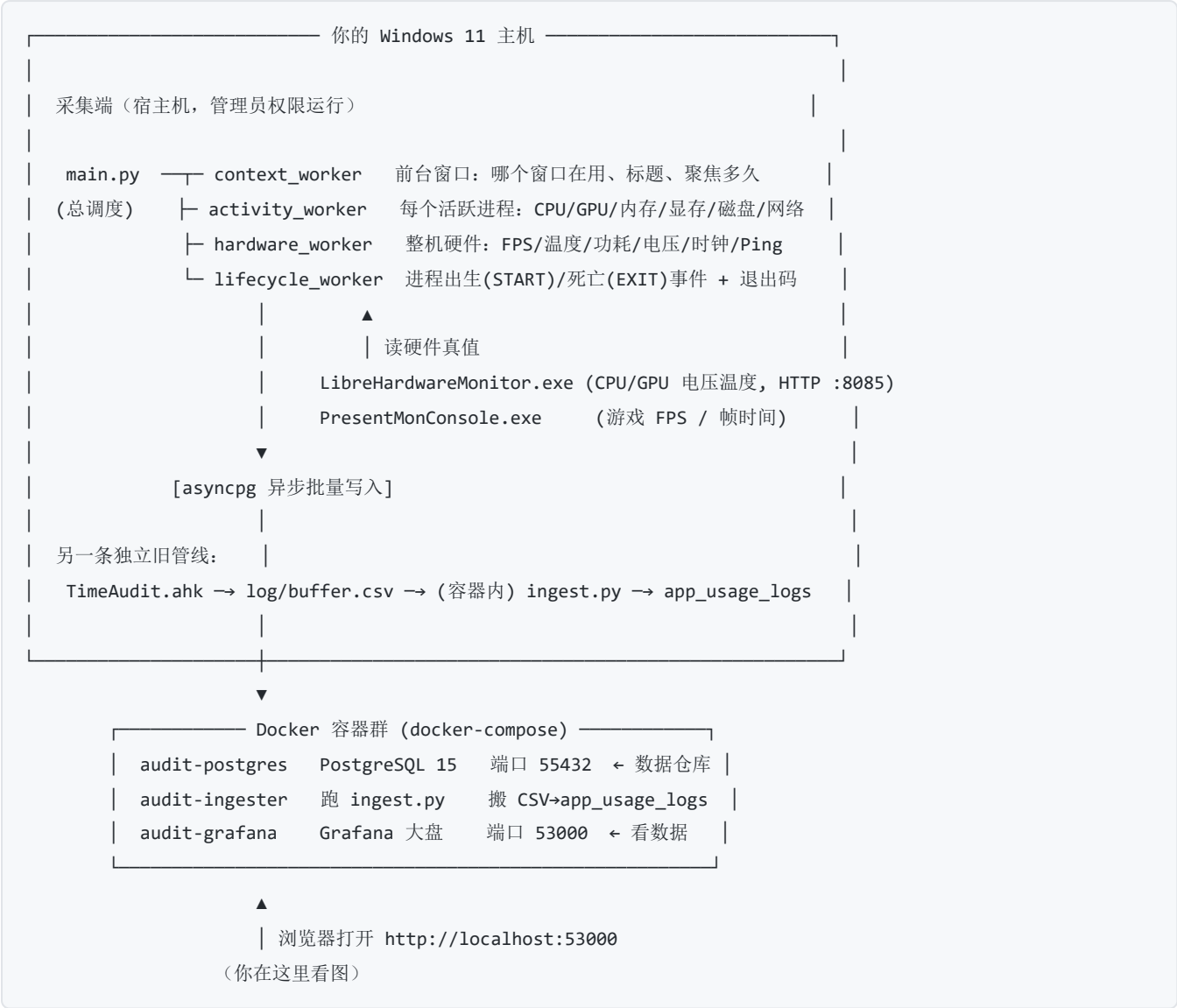
1. 一句话：它解决什么问题？

普通的任务管理器只能看"此刻"，看完就没了。**TimeAudit 的核心是"留底" + "回放"**——它把你电脑每一秒的状态都存下来，于是你可以**事后**回答这些问题：

你想知道的事	TimeAudit 怎么回答
"刚才打游戏突然卡了一下，到底谁干的？"	把时间轴拖回那 2 秒，看那一刻是哪个后台进程突然狂读硬盘 / 网络丢包 / GPU 撞了功耗墙。
"我的硬盘半夜一直在响，是谁？"	查 <code>is_foreground=0</code> 的后台进程，按磁盘读写速率排序，凶手立现。
"几十个 svchost.exe 哪个在吃 CPU？"	每个 svchost 都还原出了它背后真正的服务名（ <code>service_name</code> ）。
"这个进程是不是病毒伪装的系统文件？"	看 <code>signature_status</code> （数字签名）和 <code>command_line</code> （完整启动命令）。
"我今天到底在哪个软件上花了最多时间？"	前台聚焦时长（ <code>duration_ms</code> ）按 App 汇总。
"通宵跑 AI 训练耗了多少电？"	对 <code>gpu_board_power</code> + <code>cpu_package_power</code> 按时间积分。
"那个程序无缘无故闪退，死因是什么？"	查它的 <code>EXIT</code> 事件里的退出码（如 <code>0xC0000005</code> = 内存非法访问）。

适用场景：个人的高配工作站（本项目就是为 Windows 11 + AMD 9950X3D + RTX 5080 这套机器调的），想长期、细粒度地审计自己电脑的性能与时间花销。它**不是**给公司批量部署的监控（没有多机、没有告警推送），就是一个硬核极客的"自己电脑的飞行记录仪"。

2. 整体怎么转？（数据从哪来，到哪去）



一句话总结：采集端（Python）每 3 秒采一拍 → 直接写进 Docker 里的 PostgreSQL → 你用浏览器开 Grafana 看图。

3. ⚠ 一个容易搞混的点：这里其实有"两条"数据管线

项目历史上长出了两套前台记录，它们同时在跑、互不干扰，新手最容易在这里懵：

	管线 A：旧版「简版工时」	管线 B：主引擎「全量遥测」
谁采集	TimeAudit.ahk (AutoHotkey 脚本)	main.py + 5 个 worker (Python)
采什么	只采"前台哪个窗口、用了多久"，自带空闲/睡眠/锁屏判定	前台窗口 + 全部进程指标 + 整机硬件 + 进程生死
怎么落库	先写 log/buffer.csv，再由容器里的 ingest.py 每 10 秒搬进数据库	Python 直接异步写数据库
进哪张表	app_usage_logs (一张简单表)	fact_* / dim_* 一套分区事实表

你真正要看的、信息量最大的，是管线 B（fact_* 表）。管线 A 是更早的轻量版本，留着兜底/对照。本 README 后面讲的"采集舱"数据表"主要指管线 B。

4. 采集端：5 个 worker 各管一摊

主程序 `main.py` 是“总指挥”：它建数据库连接池、把 4 个采集 worker 拉起来，然后进入一个**每 3 秒一拍**的主循环，每拍让各 worker 采一次数据并批量写库。下面逐个说人话。

`main.py` — 总调度 + 守护外壳

- 每 3 秒一拍驱动所有采集。
- **单例锁**：保证全机只有一个引擎在跑（新实例会抢占踢掉旧的）。
- **崩溃自愈外壳**：主循环若整个崩了，外层 `while True` 会等 5 秒重启它。
- **睡眠/唤醒处理**（重点，见第 7 节）：用“墙上时间”判断系统是否刚从睡眠/休眠醒来，醒来后把跨睡眠的脏数据截断掉。
- **冷启动清理**：每次启动先把上次“关机时没来得及收尾”的前台会话补上结束时间（否则会留下永远不结束的“幽灵行”）。
- **分区预热**：每 12 小时（按墙上时间）提前把“下一周/下一月”的数据库分区建好，免得到了周一零点没表可写而丢数据。
- **日志治理**：`telemetry.log` 超过 50MB 自动清空截断。

`context_worker.py` — 前台上下文舱

- 用 Win32 API 高频问：“现在最前面的窗口是谁、标题是什么、是全屏还是窗口”。
- 窗口一换，就把上一个窗口的会话“结算”：写下它聚焦了多少毫秒（`duration_ms`）。
- 写进 `fact_process_context`。

`activity_worker.py` — 活跃进程舱

- 用 NTDLL 一次性拿到全系统进程快照（比逐个 `psutil` 快得多），算出每个进程这一拍的 CPU、磁盘读写、IOPS（都是“速率”，靠两拍差分算）。
- GPU 显存/占用走 Windows 图形内核的 PDH 计数器（和任务管理器同源），并只认 **RTX 5080 这张独显**（按厂商 ID 锁 LUID，隔离核显和虚拟显示器，见第 7 节）。
- 网络流量按“每进程连接数占比”近似分摊（这是个已知的粗略估算，不是精确值）。
- 写进 `fact_process_activity`，并维护进程身份维度表 `dim_process_registry`。

`hardware_worker.py` — 整机硬件舱

- NVML 读 GPU：利用率、温度、功耗、显存时钟、PCIe、降频原因。
- PDH 读 CPU：频率、ACPI 温度、硬缺页等。
- **LibreHardwareMonitor**（外部 exe）通过 HTTP `http://127.0.0.1:8085/data.json` 读 NVML/PDH 给不出来的真值：CPU 核心电压(Vcore)、CPU 封装温度(Tctl/Tdie)、GPU 核心电压、GPU 热点温度。
- **PresentMonConsole**（外部 exe）抓游戏的 FPS / 帧时间 / 1% Low。
- 自己测 DPC 延迟、Ping、丢包、抖动。
- 这两个外部 exe 都有**看门狗**：进程没了自动隐身重启；LHM 的网页服务卡死了也会强杀重拉。
- 写进 `fact_system_hardware`。

`lifecycle_worker.py` — 进程生死舱

- 每秒对全系统进程做一次“差分”：这一秒多出来的就是“出生(START)”，少掉的就是“死亡(EXIT)”。
- 进程出生时就提前抓住它的内核句柄，这样它死的时候才能读到**真实退出码**（如 `0xC0000005`）和存活时长。

- 顺带做"数字签名校验"（判断是不是微软官方签名）和"是否提权"。
- 写进 `fact_process_lifecycle_events`。

5. 数据库里有哪些表？（每张表存什么，大白话）

数据库名 `time_audit`，跑在 Docker 容器 `audit-postgres` 里，宿主机端口 **55432**。

表名	类型	存什么	怎么分区
<code>dim_process_registry</code>	维度表	每个"独一无二的程序身份"一行：进程名+路径+命令行+父进程+是否提权+签名状态。事实表用整数 <code>process_key</code> 指向它，省空间。	不分区
<code>fact_process_activity</code>	事实表	每 3 秒 × 每个活跃进程一行：CPU/GPU/内存/显存/磁盘/网络/线程数等。	按周
<code>fact_process_context</code>	事实表	前台窗口会话：哪个窗口、标题、 <code>duration_ms</code> （聚焦多久）。	按周
<code>fact_system_hardware</code>	事实表	每 3 秒一行整机硬件画像：FPS、CPU/GPU 温度功耗电压时钟、内存、磁盘延迟、Ping。	按月
<code>fact_process_lifecycle_events</code>	事实表	进程 START/EXIT 离散事件 + 退出码 + 存活秒数。	不分区
<code>app_usage_logs</code>	旧版表	管线 A（AHK→CSV）的简版前台工时。	不分区

几个关键字段的含义（看图时会用到）：

- `proc_cpu_usage`：整机口径 **0–100%**（已按 32 个逻辑核归一化）。不是单核百分比，所以一个多线程进程也不会超过 100%。
- `signature_status`：`1` = 有效数字签名（多半是正经软件），`0` = 没签名（可疑），`-1/-2` = 校验出错。
- `is_elevated`：`1` = 管理员权限运行，`0` = 普通，`-1/-2` = 查不到。
- `window_mode`：`2` = 全屏/无边框，`3` = 普通窗口。
- `gpu_throttling_reasons`：GPU 降频原因的二进制位（撞功耗墙/温度墙等）。
- `duration_ms`：前台窗口连续聚焦的毫秒数。**注意**：跨睡眠/锁屏的会话会被引擎截断，不会把睡觉时间算成"在用"。

完整建表语句见 `schema.sql`（全新装机时用它建表；含 `pg_trgm` 扩展与全部覆盖/局部索引）。

数据保留 / 三年可行性：`fact_process_activity` 约 2GB/周，跑满三年约 **330GB**，对 E 盘（数 TB）完全无压力，**三年内无需任何清理**。`main.py` 里有个 `RETENTION_DAYS`（默认 **1200 天 ≈ 3.3 年**）兜底：每 12 小时随分区预热顺手 `DROP` 掉上界早于保留期的旧周/月分区、并删两张非分区表的超期行——**默认值大于 3 年，所以三年内绝不触发删除**；设成 `0` 可彻底关闭、永久留全史。

6. 存储 + 展示：Docker 那三个容器

`docker-compose.yml` 定义了 3 个容器（开机由 `start_all.bat` 里的 `docker compose up -d` 拉起）：

容器	镜像	端口	干嘛
<code>audit-postgres</code>	<code>postgres:15-alpine</code>	<code>55432→5432</code>	数据仓库。数据存在宿主机 <code>./postgres_data</code> 目录。

容器	镜像	端口	干嘛
audit-ingester	本地 Dockerfile 构建	无	跑 ingest.py，每 10 秒把 log/buffer.csv 搬进 app_usage_logs。
audit-grafana	grafana-oss:latest	53000→3000	网页大盘。配置从 ./grafana_provisioning 注入，数据存 ./grafana_data。

账号/端口速查：

- PostgreSQL：localhost:55432，用户 leyang，密码 SecurePassword123，库 time_audit。
- Grafana：浏览器开 http://localhost:53000。

PostgreSQL 性能配置写在 docker-compose.yml 的 command：里（不是 postgresql.conf）：shared_buffers=2GB、work_mem=16MB、effective_cache_size=8GB，以及 NVMe 友好的 random_page_cost=1.1 / effective_io_concurrency=200；外加 shm_size: '512mb'（并行查询大分区时 /dev/shm 的上限，防 "could not resize shared memory segment ... No space left on device"）。改这些要编辑 compose 后 docker compose up -d audit-db 重建容器才生效。

容器时区锁定为 Asia/Shanghai（compose command：里的 -c timezone=Asia/Shanghai）。这是功耗大盘"今日/本周/本月"统计正确的前提——这些面板用 date_trunc('day', now()) AT TIME ZONE 'Asia/Shanghai' 当边界，若会话时区是 UTC，边界会整体偏移 8 小时（"今日能耗"会从早上 8 点才开始算）。别删这行，否则 docker compose up -d 重建后时区 bug 复现。

 Grafana 里有 3 个 PostgreSQL 数据源，全部指向同一个 time_audit 库：硬件大盘用 P7A9DAD60F8AB4C18，其余大盘用 bfoc1vymtgni8a，还有一个 provisioning 注入的 PostgreSQL 没被引用。不同大盘用不同 UID 不是 bug（都连同一个库），别去「统一」它们，否则现有面板会断。

7. 关键设计 & 千万别踩的坑（给 AI 维护者重点看）

这一节是整个项目最值钱的部分——很多写法是故意这样写的，看着别扭但有血泪原因。改代码前务必读完。

1. 睡眠/关机边界，一律用"墙上时间" time.time()，绝不用"单调时钟"。asyncio 的事件循环时钟（单调时钟 monotonic）在系统睡眠时会暂停，所以"靠单调时钟跳变来判断唤醒"是永远不会触发的死代码。现在 main.py 用墙上时间差（超过 60 秒）判断刚睡醒，醒来后会：① 把当前前台会话按"睡前那一刻"截断（否则你合盖睡 8 小时，醒来会被算成"看了这个文档 8 小时"——这是真实发生过的 24 小时脏数据）；② 重置网络/CPU 速率基线；③ 强制补建分区。
2. 网络/CPU/IO 的"速率"用单调时钟 time.monotonic()，免疫 NTP 对时回拨。墙上时间偶尔会被 NTP 往回拨几秒，如果用它算速率，分母会变成极小值导致瞬间几千倍的假尖刺。所以速率算分母用单调时钟，而落库时间戳用墙上时间（UTC）。两者分工不能混。
3. 前台会话时长 duration_ms 永远不为负。不管是正常切窗还是睡眠截断，结束时间若早于开始时间，一律收敛成 0。
4. fact_process_context 的 UPDATE 必须带 timestamp 主键。闭合前台会话的 UPDATE 语句 WHERE 里一定要带分区键 timestamp，否则会扫遍所有周/月分区（慢），而且 Windows 复用 PID 时还可能误闭合几个月前的旧行。
5. 每进程 CPU 归一化到 0-100%（除以逻辑核数并封顶）。不要改回"单核百分比"，否则多线程进程会冒出 2500% 这种吓人的值。
6. GPU 只认 RTX 5080 这张独显。这台机器有三个显示适配器（RTX5080 独显 + AMD 核显 + 向日葵虚拟显示器）。核显是 UMA 架构会把系统内存误报成几十 GB"专用显存"。所以开机时从注册表按厂商 ID(NVIDIA=0x10DE) 锁定独显的 LUID 前缀，之后每进程 GPU 数据只认这张卡。
7. NVML 的每个易失败调用要各自隔离。降频原因、PCIe 吞吐这些调用在某些驱动版本会抛异常；它们各自包了 try，单个失败不会把整块 GPU 采集拖垮、更不会触发 nvmlShutdown 把 GPU 数据全部清零。（注：throttle 接口在本机 RTX5080 上实测是支持的，不会崩。）

8. 两个外部 exe 必须保持在线，靠“看门狗重拉”而不是“降级造假”。LibreHardwareMonitor / PresentMon 掉了就自动隐身重启；LHM 网页服务卡死连续 ~15 秒也强杀重拉。理念是“要么采到真值，要么重启再采，绝不写假数据”。
9. 外部 exe 需要管理员权限。非提权环境下 PresentMon/LHM 会报 `WinError 740`，引擎会退避重试而不会崩，但拿不到 FPS/电压。所以引擎必须以管理员身份运行（开机自启任务已配好提权，见快速部署.md）。
10. 日志会自动控制大小：`telemetry.log` 超 50MB 截断；`presentmon_debug.log` 用滚动文件处理器封顶 ~15MB。
11. 进程差分扫描器的快照推进放在 `finally` 里：即使处理某个进程时抛异常，也保证基线快照前进，不会把同一个 START/EXIT 事件重复投递。
12. PG 掉线重连别用裸 `await pool.close()`。连接已死时它会等待“未释放连接”挂起 60s+（实测刷屏 `Pool.close() is taking over 60 seconds`），把整个 3 秒主循环卡死、停止写库。`main.py` 已改为 `asyncio.wait_for(pool.close(), 5s)` 超时 + 失败回退 `pool.terminate()` 强制拔线，PG 重启后秒级自愈。改重连逻辑时别把这个保护去掉。
13. `is_not_responding` 用 `IsHungAppWindow`（任务管理器同源）判定，不是 `psutil.STATUS_STOPPED`。后者是 Unix 概念、Windows 上几乎永不为真（曾导致该字段恒为 0）。现由 `activity_worker._scan_hung_pids()` 每拍 `EnumWindows` 标记消息泵卡死(>5s)的窗口属主 PID，采集时 O(1) 查表。注意它必须跑在用户交互会话里（`EnumWindows` 才看得到用户窗口），所以引擎不能用 SYSTEM 账户跑。
14. Grafana 面板 SQL 两个反复踩的坑（写大盘查询时注意）：
 - 路径判定别用 `LIKE 'c:\windows\system32%'`。PostgreSQL 的 `LIKE` 把 `\w / \s / \t` 当转义序列吃掉，路径里的反斜杠会失配——曾让“高危仿冒检测”把 172 个正常系统进程全误报。改用 `starts_with(lower(path), 'c:\windows')` 或 `position('\temp\' in lower(path))>0`（不受 `LIKE` 转义影响）。
 - `generate_series` 的时间网格起点必须对齐到桶边界。若用未对齐的 `$_timeFrom()`（带秒）生成 1 分钟网格，再去 `JOIN` 一张 `date_trunc('minute', ...)`（落在 :00）的表，两边时间戳永不相等、联结全落空——曾让“前台 vs 后台争抢”前台恒为 0。网格起点用 `date_trunc('minute', $_timeFrom())`。
15. PG 会话时区锁 `Asia/Shanghai`，“本地日界”统计才正确。功耗大盘的“今日/本周/本月”用 `date_trunc(单位, now()) AT TIME ZONE 'Asia/Shanghai'` 与 `timestamp_tz` 列比较；PG 默认会话时区是 UTC 时，无时区常量会被当 UTC 解释、边界整体偏移 8 小时。已在 `docker-compose.yml` 用 `-c timezone=Asia/Shanghai` 根治（比逐条改 78 个 SQL 更全，连“老化趋势”按日/周/月分组也一并对齐）。别删这行 `compose` 配置。注：`$_timeFilter` / `$_timeFrom` 这类 Grafana 宏传的是绝对时刻，不受会话时区影响、本来就对。
16. 每进程父进程名用「系统快照内的 pid→name 映射」解析，绝不用 `psutil.Process.parent()`。后者在 Windows 上每次调用都会全量重建 `ppid_map`（枚举所有进程）——`cProfile` 实测它占 `collect_active_processes` 总耗时的 86%（单拍约 1 秒）。`fetch_system_processes()`（`NtQuerySystemInformation`）的同一快照里早已带每个进程的 `ppid`，直接查 `pid_to_name` 即可，零额外系统调用、且同一快照内更自治。改回 `proc.parent()` 会让采集瞬间慢回 1 秒。
17. `collect_active_processes()` 在主循环里用 `await asyncio.to_thread(...)` 调，别改回同步直调。`psutil.cmdline()` 对启动中/受保护进程会触发 `ERROR_PARTIAL_COPY` 的内部重试 `sleep`（单拍累计可达 0.5–1 秒）。这是同步 `sleep`，若直接在事件循环线程里跑会冻结整个引擎（阻塞 `lifecycle` 事件处理与连接自愈）。放进工作线程后，这段 `sleep` 不再卡住 event loop。
18. `SafeStdoutWrapper` 独占锁定与多实例并发冷启动冲突。当有多个实例或后台进程（如手动启动的同时计划任务也在拉起实例）试图在非常短的短时间内同时打开 `telemetry.log` 时，会导致文件写操作的独占锁争用，抛出 `PermissionError`（拒绝访问）。主程序的 `SafeStdoutWrapper` 在类初始化中对此进行了 `try/except` 自愈防护以规避因日志句柄初始化失败导致的进程崩溃，但最佳实践仍是依赖单例锁（`enforce_singleton`）踢掉旧实例，避免多个实例长期并发双写。
19. 计划任务提权启动的 Python 解释器绝对路径与 `-WorkingDirectory` 强依赖。在 Windows 计划任务中以最高权限（Highest Privilege）拉起脚本时，由于运行环境不包含完整的用户 PATH 变量，如果使用普通 `python` 命令行，或者未显式设置 `WorkingDirectory`，经常会导致解释器闪退或因为相对路径偏移找不到外部依赖（如 `LibreHardwareMonitor.config`）。必须始终使用 Python 解释器的绝对物理路径（如

C:\Users\10979\AppData\Local\Programs\Python\Python311\pythonw.exe) , 并且在计划任务中指定 “起始于” (Start in) 为项目根目录 E:\TimeAudit 。

20. **分区大表查询强制裁剪下推 (Grafana SQL 核心调优)** 。数据库内 `fact_process_activity` (活跃进程表) 是高频时序数据, 运行三年后将积累数千万行数据。为避免全表扫描或全分区扫描拖垮数据库甚至导致 Grafana 面板超时卡死, 在编写或修改任何针对该表 (及其他分区表) 的 SQL 查询时, **必须在 WHERE 子句中显式加上时间下界** (如 `timestamp >= $__timeFrom()` 或带有明确的时间间隔偏移)。若有 JOIN 查询, 在 JOIN 条件中也必须将关联时间下界下推, 确保 PostgreSQL 的优化器百分之百进行 “分区剪裁” (Partition Pruning) 。

8. 怎么跑起来 / 怎么看数据 / 怎么排查

完整安装、换机、容灾步骤在 [快速部署.md](#)。这里只列日常最常用的几条。

看实时日志 (引擎在干嘛、谁出生谁死亡) :

```
Get-Content -Wait -Tail 20 E:\TimeAudit\telemetry.log -Encoding utf8
```

一键体检 (强烈推荐排查问题时先跑它, 会逐项检查: 组件存活 / 真值入库 / CPU 归一化 / 显存锁卡 / 分区建表 / 数据质量) :

```
python E:\TimeAudit\test_telemetry_health.py
```

跑单元测试套件 (改完代码后回归, 应 6/6 全绿) :

```
python E:\TimeAudit\telemetry_test_suite.py
```

跑优化/修复回归测试 (连接池封顶 / 数据库调优 / 无响应检测 / 前端面板回归, 应 14/14 全绿) :

```
python E:\TimeAudit\test_optimizations.py
```

手动重启引擎 (用配好的提权计划任务, 最干净) :

```
schtasks /run /tn TimeAudit_AutoStart
```

手动全量备份 (数据库 + Grafana 仪表盘) :

```
powershell -ExecutionPolicy Bypass -File E:\TimeAudit\backup_all.ps1
```

数据库和 Grafana 仪表盘每天凌晨 4 点会由计划任务 `TimeAudit_DailyBackup` 自动备份, 仪表盘还会自动 commit + push 到 GitHub。无需手动导出。详见[快速部署.md](#)。

看大盘: 浏览器开 <http://localhost:53000> 。

9. 目录结构

E:\TimeAudit\	
├─ main.py	总调度 + 3 秒主循环 + 睡眠/分区/自愈
├─ context_worker.py	前台窗口上下文舱 → fact_process_context
├─ activity_worker.py	活跃进程舱 → fact_process_activity (+ dim_process_registry)
├─ hardware_worker.py	整机硬件舱 → fact_system_hardware
├─ lifecycle_worker.py	进程生死舱 → fact_process_lifecycle_events
├─ TimeAudit.ahk	旧版前台工时脚本 (管线 A) → log/buffer.csv
├─ ingest.py	容器内: 把 buffer.csv 搬进 app_usage_logs
├─ docker-compose.yml	Postgres + Ingester + Grafana 三容器编排
├─ Dockerfile	Ingester 容器构建
├─ schema.sql	【全新装机用】建好所有父表+索引
├─ requirements.txt	宿主机 Python 依赖 (psutil / nvidia-ml-py / asyncpg)
├─ README.md	本文件: 架构 / 数据流 / 关键设计与坑 (给开发者+AI 维护者)
├─ 快速部署.md	全新安装 / 换机迁移 / 容灾恢复 / 日常运维
├─ 使用手册.md	6 张仪表盘 78 个面板逐个精讲 (给小白看数据/排查)
├─ backup_all.ps1	一键全量备份(库+大盘): 每日计划任务调用它
├─ backup_db.ps1	备份数据库 (pg_dump -Fc + 自动清旧)
├─ backup_grafana.py	备份 Grafana 仪表盘(导出JSON+git push) + grafana.db
├─ restore_grafana.py	从备份 JSON 恢复 Grafana 仪表盘
├─ start_all.bat	开机自启: 拉起 AHK + Docker + 提权的 main.py
├─ check_status_gui.ps1	图形化状态体检小工具
├─ test_telemetry_health.py	在线健康综合体检 (先跑它排查问题)
├─ telemetry_test_suite.py	单元/集成测试套件 (6 个用例)
├─ test_optimizations.py	优化/修复回归测试 (连接池/调优/无响应检测/前端, 14 用例)
├─ LibreHardwareMonitor.exe	外部硬件探针 (CPU/GPU 电压温度, HTTP :8085)
├─ PresentMonConsole.exe	外部帧率探针 (FPS / 帧时间)
├─ postgres_data/	PostgreSQL 数据卷 (别手删!)
├─ grafana_data/	Grafana 数据卷 (含 grafana.db 仪表盘库)
├─ grafana_provisioning/	Grafana 数据源/大盘 provider 配置 (自动注入容器)
├─ backups/	数据库 .dump 备份 (不进 git)
├─ grafana_dashboards/	Grafana 仪表盘 JSON = 前端"代码"(进 git, 每日自动导出+push)
├─ grafana_backups/	grafana.db 二进制库备份 = "数据"(不进 git, 轮转14份)
└─ log/	AHK 的 buffer.csv + 备份日志 backup.log

10. 给 AI 维护者的快速上手清单

1. 先读第 7 节"关键设计 & 坑", 那里是历史血泪, 照着它就不会把修好的 bug 改回去。

2. **改完代码先跑三件事**： `python telemetry_test_suite.py` (应 6/6 绿) + `python test_optimizations.py` (应 14/14 绿) + `python test_telemetry_health.py` (引擎在跑时应大部分绿)。
3. **跑测试要用 UTF-8**：默认 GBK 控制台会因日志里的 emoji 崩；套件已自愈 stdout，但你手动跑别的脚本时记得 `PYTHONUTF8=1`。
4. **这是写密集时序库**：每 3 秒几十行写入。加索引、改表结构前先想清楚写放大代价。
5. **本机就是目标机** (RTX 5080 + 9950X3D, PostgreSQL 跑在 55432)。很多硬件相关逻辑可以直接实机验证，别只靠推演。
6. **测试里 mock NVML 时，假句柄(MagicMock)千万别传给真实 NVML 函数**——会在 C 层访问违例直接把进程干崩 (try/except 拦不住)。所有 NVML 函数名都要 mock 到位。

TimeAudit 是个人项目。数据全部留在本机，不上传任何云端。